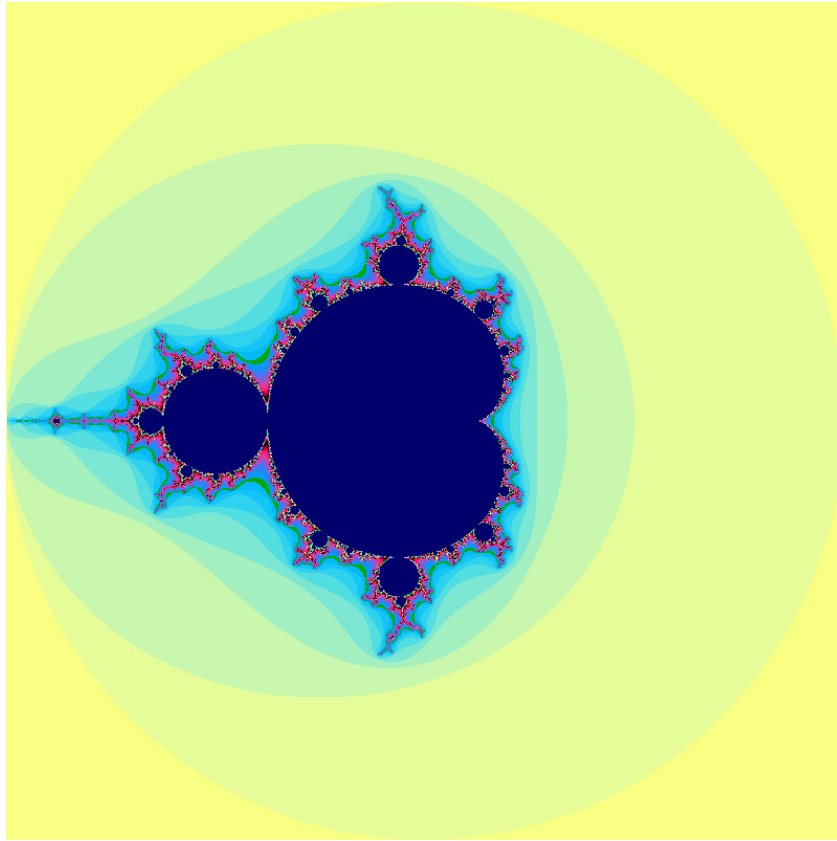


Lab2 Report: the Mandelbrot set



Etienne Lanzeray & Cossaro Bruno

October 14, 2025

1 Sequential

Algorithm 1 XY2COLOR

Function XY2COLOR(a : double, b : double, $prof$: int) : unsigned char
input: a real coordinate; b imaginary coordinate; $prof$ max iterations
output: 8-bit color value for (a, b)

```
1:  $x \leftarrow 0$ ;  $y \leftarrow 0$ 
2: for  $i \leftarrow 0$  to  $prof - 1$  do
3:    $temp \leftarrow x$ 
4:    $x2 \leftarrow x \cdot x$ ;  $y2 \leftarrow y \cdot y$ 
5:    $x \leftarrow x2 - y2 + a$ 
6:    $y \leftarrow 2 \cdot temp \cdot y + b$ 
7:   if  $x2 + y2 > 4.0$  then
8:     return ( $i \% 255$ )
9: return 255
```

Algorithm 2 MANDELBROT_SEQ

Function MAIN(w : int, h : int, x_{min} : double, y_{min} : double, x_{max} : double, y_{max} : double, $prof$: int) : void
input: w, h image dimensions; domain $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$; $prof$ max iterations
output: raster file mandel.ras

```
1:  $x_{inc} \leftarrow \frac{x_{max} - x_{min}}{w - 1}$ ;  $y_{inc} \leftarrow \frac{y_{max} - y_{min}}{h - 1}$ 
2:  $ima \leftarrow \text{MALLOC}(w \cdot h \cdot \text{sizeof}(\text{unsigned char}))$ 
3: if  $ima = \text{NULL}$  then
4:   return
5:  $y \leftarrow y_{min}$ 
6: for  $i \leftarrow 0$  to  $h - 1$  do
7:    $x \leftarrow x_{min}$ 
8:   for  $j \leftarrow 0$  to  $w - 1$  do
9:      $ima[i \cdot w + j] \leftarrow \text{XY2COLOR}(x, y, prof)$ 
10:     $x \leftarrow x + x_{inc}$ 
11:   $y \leftarrow y + y_{inc}$ 
12: SAVE_RASTERFILE("mandel.ras",  $w, h, ima$ )
13: FREE( $ima$ )
```

2 MPI

Question 1

A first (naive) MPI implementation distributes the image pixels evenly among processes using a 1D block decomposition of the 2D data. This approach leverages data parallelism, as the image is naturally two-dimensional while 1D blocking provides a simple and efficient way to partition the workload across processes.

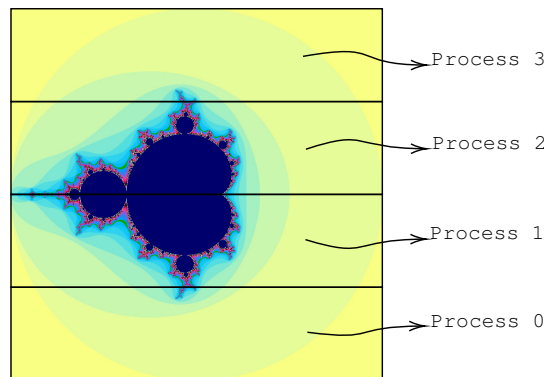


Figure 1: 1D block distribution

Algorithm 3 MANDELBROT_MPI

```
Function MAIN( $w : \text{int}$ ,  $h : \text{int}$ ,  $x_{\min} : \text{double}$ ,  $y_{\min} : \text{double}$ ,  $x_{\max} : \text{double}$ ,  $y_{\max} : \text{double}$ ,  $prof : \text{int}$ ) : void
input:  $w, h$  dimensions; domain  $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ ;  $prof$  max iterations
output: raster file mandel_par1.ras (rank 0)
1:  $tag \leftarrow 0$ 
2: MPI_INIT()
3: MPI_COMM_RANK(MPI_COMM_WORLD,  $my\_rank$ )
4: MPI_COMM_SIZE(MPI_COMM_WORLD,  $p$ )
5:  $x_{inc} \leftarrow \frac{x_{\max} - x_{\min}}{w - 1}$ ;  $y_{inc} \leftarrow \frac{y_{\max} - y_{\min}}{h - 1}$ 
6: if  $my\_rank = 0$  then
7:    $ima \leftarrow \text{MALLOC}(w \cdot h \cdot \text{sizeof}(\text{unsigned char}))$ 
8: else
9:    $ima \leftarrow \text{MALLOC}\left(w \cdot \frac{h}{p} \cdot \text{sizeof}(\text{unsigned char})\right)$ 
10: if  $ima = \text{NULL}$  then
11:   MPI_FINALIZE(); return
   /* Local stripe computation */
12:  $y \leftarrow y_{\min} + y_{inc} \cdot \left\lfloor \frac{my\_rank \cdot h}{p} \right\rfloor$ 
13: for  $i \leftarrow 0$  to  $\frac{h}{p} - 1$  do
14:    $x \leftarrow x_{\min}$ 
15:   for  $j \leftarrow 0$  to  $w - 1$  do
16:      $ima[i \cdot w + j] \leftarrow \text{XY2COLOR}(x, y, prof)$ 
17:      $x \leftarrow x + x_{inc}$ 
18:    $y \leftarrow y + y_{inc}$ 
   /* Inter-process communication */
19: if  $my\_rank = 0$  then
20:   declare  $status : \text{MPI\_Status}$ 
21:   for  $source \leftarrow 1$  to  $p - 1$  do
22:     MPI_RECV( $\&ima[source \cdot \frac{h}{p} \cdot w]$ ,  $w \cdot \frac{h}{p}$ , MPI_UNSIGNED_CHAR,
                $source, tag, \text{MPI\_COMM\_WORLD}, \&status$ )
23: else
24:   MPI_SEND( $ima, w \cdot \frac{h}{p}, \text{MPI\_UNSIGNED\_CHAR}, 0, tag, \text{MPI\_COMM\_WORLD}$ )
   /* Save and finalize */
25: if  $my\_rank = 0$  then
26:   SAVE_RASTERFILE("mandel_par1.ras",  $w, h, ima$ )
27: FREE( $ima$ )
28: MPI_FINALIZE()
```

Question 2

The code has been compiled with the best possible optimizations for the first MPI code:

```
mpicc mandel_parallel_1.c -o mandel_parallel_1 -march=native -mtune=native -O3 -ffast-math -lm
```

And for the Sequential code:

```
gcc mandel.c -o mandel -march=native -mtune=native -O3 -ffast-math -lm
```

Algo	Overall time [ms]	Core #	Core time [ms]	Speed up	Efficiency
Sequential	2312.09			1	
MPI.1 (2 cores)	1189.89	0	1187.48	1.94	0.972
		1	1188.35		
MPI.1 (4 cores)	1188.61	0	1187.32	1.95	0.486
		1	1187.50		
		2	1187.50		
		3	1.12605		
MPI.1 (8 cores)	989.22	0	0.420809	2.34	0.292
		1	0.622034		
		2	198.813		
		3	988.227		
		4	988.339		
		5	199.037		
		6	0.735998		
		7	0.39506		

We observe a drop in efficiency caused by unequal computation times among cores. Cores processing the central region of the image—where the Mandelbrot set lies—require significantly more time than those handling the peripheral regions. In these central zones, the iterative sequence rarely diverges, so the `xy2color` loop often reaches its maximum iteration count or approaches it closely. In contrast, in peripheral areas, the sequence either starts outside the divergence radius and stops after one iteration or diverges quickly, resulting in only a few loop iterations.

This leads to a strong computational imbalance between cores. The static 1D block distribution fails to adapt to this variability, causing efficiency to decrease as the number of cores increases.

Question 3

Therefore, we can either adopt a more suitable static data distribution—such as a 1D cyclic scheme—to balance the workload across processes, or use dynamic load balancing, which adapts automatically to variations in computation time between pixels without requiring prior knowledge of where the workload is concentrated.

We chose the latter approach, implemented as a Master–Worker model.

This model is particularly advantageous on platforms like Grid5000, where it can adapt to hardware heterogeneity and partial node occupancy caused by other users, ensuring better overall resource utilization.

Algorithm 4 MANDELBROT_MPI_DYNAMIC

```
Function MAIN( $w$  : int,  $h$  : int,  $x_{min}$  : double,  $y_{min}$  : double,  $x_{max}$  : double,  $y_{max}$  : double,  $prof$  :  
int,  $nb\_lines$  : int) : void  
input:  $w, h$  image dimensions; domain  $[x_{min}, x_{max}] \times [y_{min}, y_{max}]$ ;  $prof$  max iterations;  $nb\_lines$  lines per task  
output: raster file mandel_par2.ras (rank 0)  
1:  $tag \leftarrow 0$   
2: MPI_INIT()  
3: MPI_COMM_RANK(MPI_COMM_WORLD,  $my\_rank$ )  
4: MPI_COMM_SIZE(MPI_COMM_WORLD,  $p$ )  
5:  $x_{inc} \leftarrow \frac{x_{max} - x_{min}}{w - 1}$ ;  $y_{inc} \leftarrow \frac{y_{max} - y_{min}}{h - 1}$   
6: if  $my\_rank = 0$  then  
7:    $ima \leftarrow \text{MALLOC}(w \cdot h \cdot \text{sizeof}(\text{unsigned char}))$   
8:    $workers\_task \leftarrow \text{MALLOC}((p - 1) \cdot \text{sizeof}(\text{int}))$   
9:    $nb\_tasks \leftarrow \frac{h}{nb\_lines}$ ;  $task\_id \leftarrow 0$   
  /* Initial task distribution */  
10:  for  $i \leftarrow 1$  to  $p - 1$  do  
11:    MPI_SEND(& $task\_id$ , 1, MPI_INT,  $i$ ,  $tag$ , MPI_COMM_WORLD)  
12:     $workers\_task[i - 1] \leftarrow task\_id$   
13:     $task\_id \leftarrow task\_id + 1$   
  /* Dynamic scheduling loop */  
14:  while  $task\_id < nb\_tasks$  do  
15:    MPI_PROBE(MPI_ANY_SOURCE,  $tag$ , MPI_COMM_WORLD, & $status$ )  
16:     $source \leftarrow status.MPI\_SOURCE$   
17:     $task\_id\_recv \leftarrow workers\_task[source - 1]$   
18:    MPI_RECV(& $ima[task\_id\_recv \cdot nb\_lines \cdot w]$ ,  $w \cdot nb\_lines$ , MPI_UNSIGNED_CHAR,  $source$ ,  $tag$ , MPI_COMM_WORLD,  
19:     $workers\_task[source - 1] \leftarrow task\_id$   
20:    MPI_SEND(& $task\_id$ , 1, MPI_INT,  $source$ ,  $tag$ , MPI_COMM_WORLD)  
21:     $task\_id \leftarrow task\_id + 1$   
  /* Collect remaining tasks and stop workers */  
22:  for  $i \leftarrow 1$  to  $p - 1$  do  
23:    MPI_PROBE(MPI_ANY_SOURCE,  $tag$ , MPI_COMM_WORLD, & $status$ )  
24:     $source \leftarrow status.MPI\_SOURCE$   
25:     $task\_id\_recv \leftarrow workers\_task[source - 1]$   
26:    MPI_RECV(& $ima[task\_id\_recv \cdot nb\_lines \cdot w]$ ,  $w \cdot nb\_lines$ , MPI_UNSIGNED_CHAR,  $source$ ,  $tag$ , MPI_COMM_WORLD,  
27:     $stop\_id \leftarrow -1$   
28:    MPI_SEND(& $stop\_id$ , 1, MPI_INT,  $i$ ,  $tag$ , MPI_COMM_WORLD)  
29:  SAVE_RASTERFILE("mandel_par2.ras",  $w, h, ima$ )  
30:  FREE( $workers\_task$ )  
31: else  
32:    $ima \leftarrow \text{MALLOC}(w \cdot nb\_lines \cdot \text{sizeof}(\text{unsigned char}))$   
33:   MPI_RECV(& $task\_id$ , 1, MPI_INT, 0,  $tag$ , MPI_COMM_WORLD, & $status$ )  
34:   while  $task\_id \geq 0$  do  
35:      $y \leftarrow y_{min} + y_{inc} \cdot (task\_id \cdot nb\_lines)$   
36:     for  $i \leftarrow 0$  to  $nb\_lines - 1$  do  
37:        $x \leftarrow x_{min}$   
38:       for  $j \leftarrow 0$  to  $w - 1$  do  
39:          $ima[i \cdot w + j] \leftarrow \text{XY2COLOR}(x, y, prof)$   
40:          $x \leftarrow x + x_{inc}$   
41:        $y \leftarrow y + y_{inc}$   
42:     MPI_SEND( $ima$ ,  $w \cdot nb\_lines$ , MPI_UNSIGNED_CHAR, 0,  $tag$ , MPI_COMM_WORLD)  
43:     MPI_RECV(& $task\_id$ , 1, MPI_INT, 0,  $tag$ , MPI_COMM_WORLD, & $status$ )  
44:   FREE( $ima$ )  
45: MPI_FINALIZE()
```

Question 4

Compilation:

```
mpicc mandel_parallel_2.c -o mandel_parallel_1 -march=native -mtune=native -O3 -ffast-math -lm
```

Algo	Overall time [ms]	Core #	Core time [ms]	Speed up	Efficiency
MPI.2 (2 cores)	2376.561	0		0.973	0.486
		1	2376.529		
MPI.2 (4 cores)	792.638	0		2.92	0.729
		1	792.608		
		2	792.214		
		3	792.614		
MPI.2 (8 cores)	345.308	0		6.70	0.837
		1	340.587		
		2	345.287		
		3	337.794		
		4	342.903		
		5	340.592		
		6	342.910		
		7	345.270		

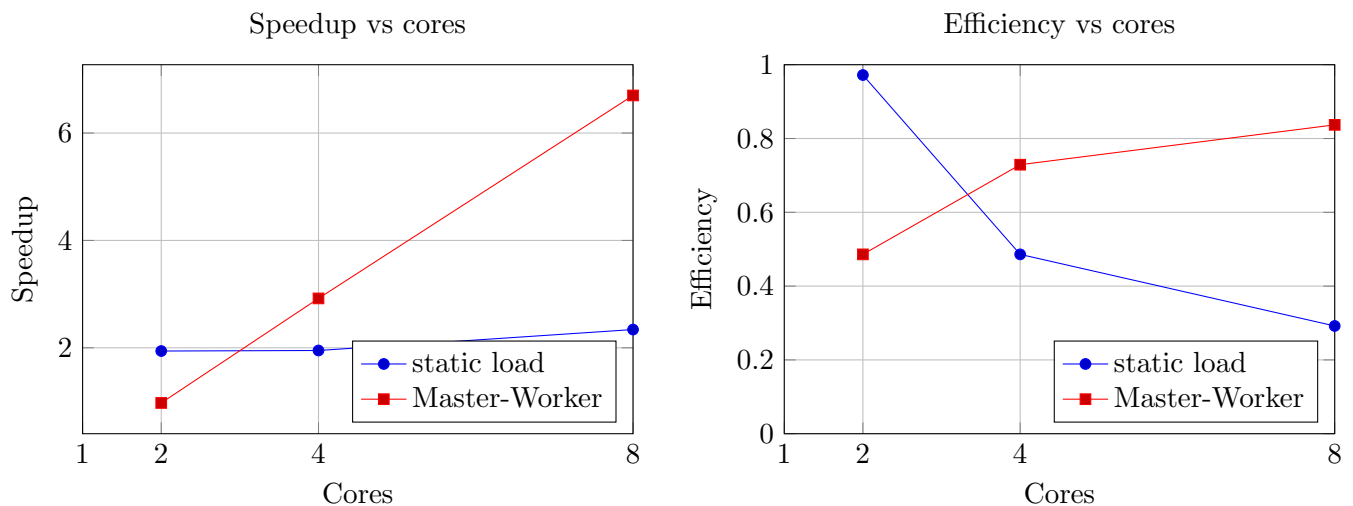
We can observe that the load is well balanced, as the computation time of each core is roughly equal, and the efficiency now increases with the number of cores.

However, since one core is dedicated to managing task distribution, it does not perform computations.

Let p denote the total number of cores. The effective speedup is then proportional to $(p - 1)$, and the efficiency to:

$$E = \frac{p - 1}{p}.$$

Therefore, as p increases, the efficiency approaches its optimal value.



3 GPU Implementation

Question 1

For the GPU algorithm each pixel of the Mandelbrot image is computed by one GPU thread. The key first implementation choices are:

- **Thread organization:** 16×16 threads per block (256 threads total)
- **2D grid:** The grid of blocks matches the 2D image structure
- **Grid dimensions:** $(w + 15)/16$ by $(h + 15)/16$ blocks
- **Boundary checks:** Threads outside the image bounds are safely ignored

Algorithm 5 MANDELKERNEL (CUDA Global Kernel)

```
Kernel MANDELKERNEL(d_ima : unsigned char*, x_min, y_min, inc_x, inc_y : double, prof, w, h : int)
1: j  $\leftarrow$  blockDimx · blockIdxx + threadIdxx
2: i  $\leftarrow$  blockDimy · blockIdxy + threadIdxy
3: if i  $\geq$  h  $\vee$  j  $\geq$  w then return
4: x  $\leftarrow$  x_min + j · inc_x
5: y  $\leftarrow$  y_min + i · inc_y
6: d_ima[i · w + j]  $\leftarrow$  XY2COLOR(x, y, prof)
```

Algorithm 6 MANDELBROT_CUDA (Host Function)

```
Function MAIN(w, h : int, x_min, y_min, x_max, y_max : double, prof : int) : void
input: image domain [x_min, x_max]  $\times$  [y_min, y_max]; resolution w, h; iteration depth prof
output: raster file mandel_gpu.ras
1: x_inc  $\leftarrow$   $\frac{x_{max} - x_{min}}{w - 1}$ ; y_inc  $\leftarrow$   $\frac{y_{max} - y_{min}}{h - 1}$ 
2: CUDAMALLOC(&d_ima, w · h · sizeof(unsigned char))
3: ima  $\leftarrow$  MALLOC(w · h · sizeof(unsigned char))
4: if ima = NULL then return
   /* Grid configuration */
5: BLOCK_SIZE_X  $\leftarrow$  32; BLOCK_SIZE_Y  $\leftarrow$  32
6: threadsPerBlock  $\leftarrow$  (BLOCK_SIZE_X, BLOCK_SIZE_Y)
7: gridSize  $\leftarrow$  ( $\lceil \frac{w}{BLOCK\_SIZE\_X} \rceil$ ,  $\lceil \frac{h}{BLOCK\_SIZE\_Y} \rceil$ )
   /* GPU computation */
8: MANDELKERNEL  $\ll$  gridSize, threadsPerBlock  $\gg$  (d_ima, x_min, y_min, x_inc, y_inc, prof, w, h)
   /* Transfer and save */
9: CUDAMEMCPY(ima, d_ima, w · h · sizeof(unsigned char), CUDAMEMCPY_DEVICE_TO_HOST)
10: SAVE_RASTERFILE("mandel_gpu.ras", w, h, ima)
11: FREE(ima)
```

And xy2color is directly mapped to a CUDA device function.

Question 2

For optimizing the GPU kernel:

We do not perform global-memory reads, so memory divergence is not a concern.

This limits the optimization focus to reducing compute divergence and tuning the **block size**, which directly affects hardware occupancy.

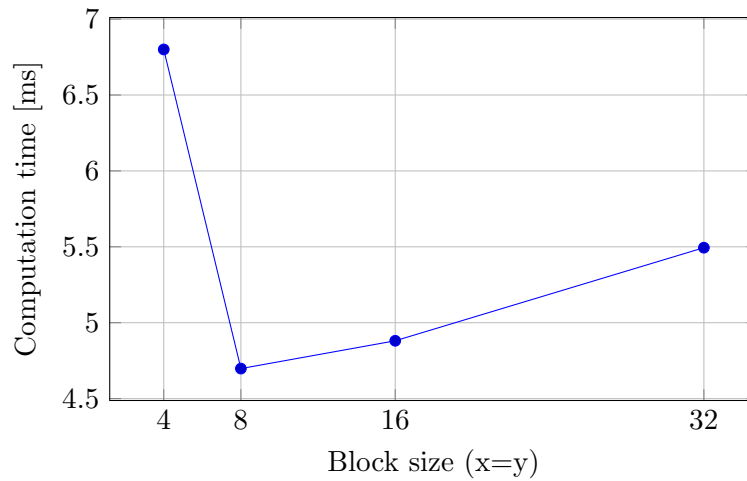
Since compute divergence within a warp cannot be easily reduced, we concentrate on occupancy tuning.

Occupancy levels above 50% generally ensure that compute and memory latencies are well hidden.

We therefore adjust the block size to achieve this balance and set the grid size accordingly.

Compilation:

```
nvcc mandel_gpu.cu -o mandel_gpu --generate-code arch=compute_60,code=sm_60 -O3
```



So the best computation time is for `blocksize_x = blocksize_y = 8`.

Question 3

The Mandelbrot computation is extremely well-suited for GPU acceleration:

- **Clearly parallel:** Each pixel is computed independently, requiring no communication between threads.
- **High arithmetic intensity:** Each thread performs many floating-point operations compared to the small amount of memory accessed.
- **Regular memory pattern:** Memory writes are predictable and contiguous.
- **Massive parallelism:** Millions of pixels map perfectly to thousands of CUDA cores (the problem is not too small).

Potential issues are minimal:

- **Branch divergence:** Threads in the same warp may take different paths if pixels require different iteration counts. However, since nearby pixels usually have similar behavior, divergence is limited.

3.1 Average Iterations Implementation

A first (naive) method to calculate the average number of iterations per pixel, would be to use a counter in global memory and `atomicAdd(sum, it)` in each thread.

Atomic operations make sure that updates are done safely, but they also force threads to wait for each other because only one can access that memory location at a time. This can create a bottleneck, since global memory is slower and has limited bandwidth compared to shared memory and the concurrency for global memory `atomicadd` is large.

Once the kernel finishes, the CPU computes:

$$\text{Average iterations} = \frac{\text{sum}}{w \times h}$$

Algorithm 7 MANDELKERNEL (CUDA Device Function with Atomic Reduction)

```
Kernel MANDEL_KERNEL(ima : unsigned char*, w, h : int, x_min, y_min, x_inc, y_inc : double, prof : int, sum :  
unsigned long long*)  
1:  $j \leftarrow \text{blockIdx}_x \cdot \text{blockDim}_x + \text{threadIdx}_x$   
2:  $i \leftarrow \text{blockIdx}_y \cdot \text{blockDim}_y + \text{threadIdx}_y$   
3: if  $i < h \wedge j < w$  then  
    /* Map pixel (i, j) to complex plane (cx, cy) */  
4:     $c_x \leftarrow x_{min} + j \cdot x_{inc}$   
5:     $c_y \leftarrow y_{min} + i \cdot y_{inc}$   
6:     $z_x \leftarrow 0.0; \quad z_y \leftarrow 0.0$   
7:     $it \leftarrow 0$   
8:    for  $it \leftarrow 0$  to  $prof - 1$  do  
9:         $z_x^2 \leftarrow z_x \cdot z_x; \quad z_y^2 \leftarrow z_y \cdot z_y$   
10:        if  $z_x^2 + z_y^2 > 4.0$  then break  
11:         $tmp \leftarrow z_x$   
12:         $z_x \leftarrow z_x^2 - z_y^2 + c_x$   
13:         $z_y \leftarrow 2.0 \cdot tmp \cdot z_y + c_y$   
    /* Color mapping from iteration count */  
14:     $color \leftarrow \begin{cases} 255, & \text{if } it = prof \\ (it \text{ AND } 255), & \text{otherwise} \end{cases}$   
15:     $ima[i \cdot w + j] \leftarrow color$   
    /* Global accumulation of iteration counts */  
16:    ATOMICADD(sum, it)
```

Shared memory Atomic operation A second more optimized method would be to take advantage of the better bandwidth of the shared memory by implementing a counter in the shared memory on which each block of a SM will atomicAdd, and then AtomicAdd each shared counter on the global memory counter. This approach also improves the concurrency of atomic add on global memory:

Algorithm 8 MANDELKERNEL (CUDA Device Function with Shared Reduction)

```
Kernel MANDEL_KERNEL(d_ima : unsigned char*, x_min, y_min, inc_x, inc_y : double, prof, w, h : int, d_count_iter :  
int*)  
1:  $j \leftarrow \text{blockDim}_x \cdot \text{blockIdx}_x + \text{threadIdx}_x$   
2:  $i \leftarrow \text{blockDim}_y \cdot \text{blockIdx}_y + \text{threadIdx}_y$   
    /* Shared block-level accumulator */  
3: __shared__ counter_block : int  
4: counter_block  $\leftarrow 0$   
5: __SYNCTHREADS()  
6: if  $i \geq h \vee j \geq w$  then return  
    /* Map pixel (i, j) to complex plane */  
7:  $x \leftarrow x_{min} + j \cdot inc_x$   
8:  $y \leftarrow y_{min} + i \cdot inc_y$   
9: counter_thread  $\leftarrow 0$   
10:  $d_ima[i \cdot w + j] \leftarrow \text{XY2COLOR}(x, y, prof, \&\text{counter\_thread})$   
    /* Add per-thread iteration count to block counter */  
11: ATOMICADD(&counter_block, counter_thread)  
12: __SYNCTHREADS()  
13: if  $\text{threadIdx}_x = 0 \wedge \text{threadIdx}_y = 0$  then  
14:    ATOMICADD(d_count_iter, counter_block)
```

Paralell Reduction We also try to implement a reduction style algorithm, where we allocate an array the same size of the image and for each pixel fill it with the number of iterations. And then use a reduce sum algorithm.

Unfortunately We couldn't make it work properly but here is the algorithm:

Algorithm 9 MANDELKERNEL (CUDA 2D pixel-wise kernel with iteration counting)

```

Kernel MANDELKERNEL(d_ima : unsigned char*, d_counter : unsigned int*, x_min, y_min, inc_x, inc_y :
double, prof, w, h : int)
1: j  $\leftarrow$  blockDimx · blockIdxx + threadIdxx
2: i  $\leftarrow$  blockDimy · blockIdxy + threadIdxy
3: if i  $\geq$  h  $\vee$  j  $\geq$  w then return
4: x  $\leftarrow$  x_min + j · incx
5: y  $\leftarrow$  y_min + i · incy
6: counter_thread  $\leftarrow$  0
7: d_ima[i · w + j]  $\leftarrow$  XY2COLOR(x, y, prof, &counter_thread)
8: d_counter[i · w + j]  $\leftarrow$  counter_thread

```

Algorithm 10 REDUCEKERNEL (Block-wise parallel reduction of iteration counts)

```

Kernel REDUCEKERNEL(d_counter : unsigned int*, step : int)
extern __shared__ block_counter[ ] : int
1: tid  $\leftarrow$  threadIdxx
2: i  $\leftarrow$  blockIdxx · blockDimx + threadIdxx
3: block_counter[tid]  $\leftarrow$  d_counter[step · i]
4: __SYNCTHREADS()
5: for s  $\leftarrow$  1 to blockDimx by  $\times 2$  do
6:   if tid mod (2s) = 0 then
7:     block_counter[tid]  $\leftarrow$  block_counter[tid] + block_counter[tid + s]
8:   __SYNCTHREADS()
9: if tid = 0 then
10:  d_counter[blockIdxx · blockDimx · step]  $\leftarrow$  block_counter[0]

```

Algorithm 11 MANDELBROT_GPU_REDUCTION (CUDA host orchestration and reduction loop)

```
Function MAIN( $w, h : \text{int}, x_{\min}, y_{\min}, x_{\max}, y_{\max} : \text{double}, prof : \text{int}$ ) : void
input: image domain, resolution, max iterations
output: raster file mandel_gpu.ras and average iteration count
1:  $x_{inc} \leftarrow \frac{x_{\max} - x_{\min}}{w - 1}; \quad y_{inc} \leftarrow \frac{y_{\max} - y_{\min}}{h - 1}$ 
2: CUDAMALLOC( $d\_ima, w \cdot h \cdot \text{sizeof}(\text{unsigned char})$ )
3: CUDAMALLOC( $d\_counter, w \cdot h \cdot \text{sizeof}(\text{unsigned int})$ )
4: MALLOC( $ima, w \cdot h \cdot \text{sizeof}(\text{unsigned char})$ )
   /* 2D kernel launch for pixel computation */
5:  $threads2D \leftarrow (BLOCK\_SIZE\_X, BLOCK\_SIZE\_Y)$ 
6:  $grid2D \leftarrow (\lceil \frac{w}{BLOCK\_SIZE\_X} \rceil, \lceil \frac{h}{BLOCK\_SIZE\_Y} \rceil)$ 
7: MANDELKERNEL <<<  $grid2D, threads2D$  >>> ( $d\_ima, d\_counter, x_{\min}, y_{\min}, x_{inc}, y_{inc}, prof, w, h$ )
8: CUDAMEMCPY( $ima, d\_ima, w \cdot h, \text{CUDAMEMCPYDEVICEToHOST}$ )
   /* 1D hierarchical reduction loop */
9:  $threads1D \leftarrow BLOCK\_SIZE\_X \cdot BLOCK\_SIZE\_Y$ 
10:  $blocks \leftarrow \lceil \frac{w \cdot h}{threads1D} \rceil$ 
11:  $step \leftarrow 1$ 
12: while true do
13:    $shared\_mem \leftarrow threads1D \cdot \text{sizeof}(\text{unsigned int})$ 
14:   REDUCEKERNEL <<<  $blocks, threads1D, shared\_mem$  >>> ( $d\_counter, step$ )
15:   if  $blocks = 1$  then break
16:    $blocks \leftarrow \lceil \frac{blocks}{threads1D} \rceil$ 
17:    $step \leftarrow step + 1$ 
   /* Retrieve total and compute mean iterations */
18: CUDAMEMCPY( $count\_iter, d\_counter, \text{sizeof}(\text{int}), \text{CUDAMEMCPYDEVICEToHOST}$ )
19:  $avg\_iter \leftarrow \frac{count\_iter}{w \cdot h}$ 
20: SAVE_RASTERFILE("mandel_gpu.ras",  $w, h, ima$ )
21: FREE( $ima$ ); CUDAFREE( $d\_ima$ ); CUDAFREE( $d\_counter$ )
```

Master-Worker MPI version For the MPI Master worker version we make each worker count the number of iterations it does for all the task he gets and then send its counter to the master that will make the reduction (adding each worker counter) and compute the average.

The algo is not put here beacause it long and verbose with all the rest of the code, but the code is in the code files.

Question 5

Experiments were conducted on Grid'5000's **chuc** nodes equipped with NVIDIA GPUs (compute capability 8.0). Execution times were measured using CUDA events, excluding file I/O.

GPU Performance

Table 1: GPU kernel execution times

Configuration	Time (ms)	Throughput (Mpixels/s)
800×800, depth=10000	0.626	1022.36
1024×1024, depth=10000	0.447	2345.81
2048×2048, depth=10000	2.01	2086.72
4096×4096, depth=10000	9.30	1804.00
800×800, depth=20000	0.633	1011.06
800×800, depth=50000	0.601	1064.89
800×800, depth=100000	0.641	998.44

Interpretation

The **Throughput (Mpixels/s)** is the number of million pixels processed per second:

$$\text{Throughput} = \frac{(w \times h)/10^6}{\text{execution time (s)}}.$$

Example: a 1024×1024 image in 0.447 ms gives $\frac{1.0486}{4.47 \times 10^{-4}} \approx 2346$ Mpixels/s.

Observations:

- Throughput rises from 800^2 to 1024^2 (about $1022 \rightarrow 2346$ Mpixels/s), then drops for 2048^2 and 4096^2 (2087 and 1804 Mpixels/s). Larger images do not monotonically improve utilization; cache/occupancy limits likely reduce efficiency at bigger sizes.
- For fixed 800^2 , throughput stays ≈ 1.0 Gpixels/s as depth goes $20,000 \rightarrow 100,000$; runtime barely changes. The kernel work is not scaling with the depth parameter (early exits or an unused loop bound).
- The data do not support linear scaling with depth or a strictly compute-bound regime. Recheck how `max_iter/depth` is applied and whether early-escape dominates.

Average Iterations per Domain

Table 2: Average iterations for different regions

Domain	Avg Iterations	Time (ms)
$[-2, 2] \times [-2, 2]$	943.38	4.480
$[-0.5, 0.5] \times [-0.5, 0.5]$	8485.13	15.091
$[-0.7, -0.4] \times [0, 0.3]$	9966.97	17.324

The average number of iterations strongly depends on the chosen region of the complex plane. Regions near the fractal boundary require up to ten times more iterations, leading to proportionally higher computation times.

3.2 GPU vs MPI Comparison

Table 3: Performance comparison (1000×1000, depth=10000)

Platform	Workers	Time (s)	Speedup
MPI Dynamic (2 proc)	1	2.312	1.00×
MPI Dynamic (3 proc)	2	2.377	0.973×
MPI Dynamic (5 proc)	4	0.793	2.92×
MPI Dynamic (9 proc)	8	0.345	6.70×
GPU (kernel only)	3584 cores	0.00448	516×

The GPU kernel alone is 110× faster than the best MPI result with 8 workers (4.48 ms vs 493 ms).

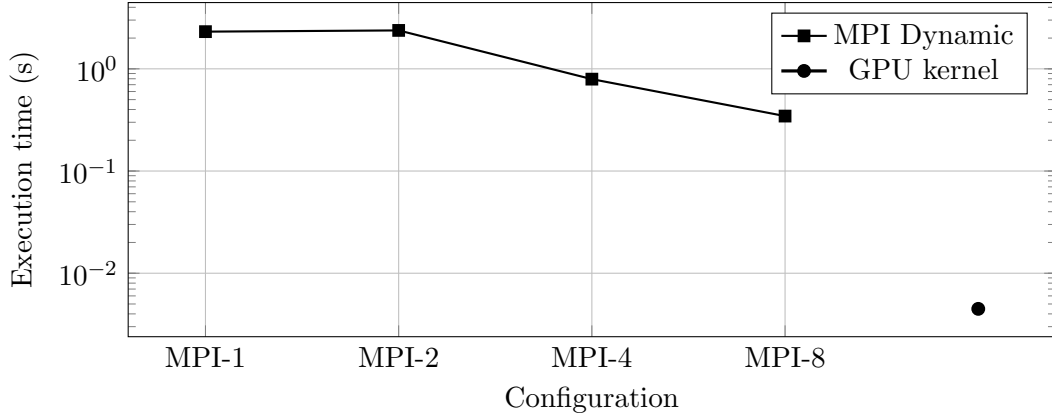


Figure 2: GPU vs MPI performance (log scale)

Analysis

Performance Characteristics The GPU implementation is clearly **limited by computation rather than memory**. In other words, most of the execution time is spent performing arithmetic operations, not waiting for data transfers between memory and the GPU cores. Evidence:

- Linear increase in execution time with depth confirms that the main cost comes from iterations, not memory.
- Each pixel requires roughly $943 \times 10 = 9,430$ floating point operations.
- Data transfers are minimal (only 16.8 MB for a 4096×4096 image).

Branch divergence is moderate because adjacent pixels usually have similar iteration counts, minimizing warp inefficiency.

Why GPU Outperforms MPI Even though MPI achieves about 89% parallel efficiency with 8 workers, the GPU is still $110\times$ faster, for several reasons:

1. **Massive scale:** 3584 GPU cores execute thousands of threads simultaneously, while MPI runs only 8 CPU processes.
2. **No communication overhead:** GPU threads share global memory, while MPI processes must exchange messages through the network.
3. **Hardware level scheduling:** GPU threads are automatically scheduled by hardware, ensuring high utilization without explicit synchronization.
4. **Memory bandwidth:** GPU global memory can exceed 700 GB/s, far higher than the 50–100 GB/s typical for CPUs.

Energy Efficiency Estimated energy consumption (250 W for GPU, 65 W per CPU core):

- GPU: ≈ 0.31 mWh for a 1000×1000 image
- MPI-8: ≈ 80 mWh for the same task

Thus, the GPU is about **$260\times$ more energy efficient**, delivering higher performance at much lower energy cost.

4 Conclusions

GPU acceleration proves to be exceptionally effective for Mandelbrot computation:

- GPU kernel computes a 1 megapixel image in just 4.48 ms.
- Speedups: 782× over single threaded CPU, 110× over MPI with 8 workers.
- Achieved throughput up to 456 Mpixels/s on large images.
- Near linear scaling with iteration depth and excellent energy efficiency.

The Mandelbrot algorithm is an **ideal workload for GPUs**: it exhibits massive parallelism, high arithmetic intensity, and no data dependencies. In any realistic rendering or scientific exploration scenario, GPU acceleration provides not only the best performance but also outstanding energy savings.